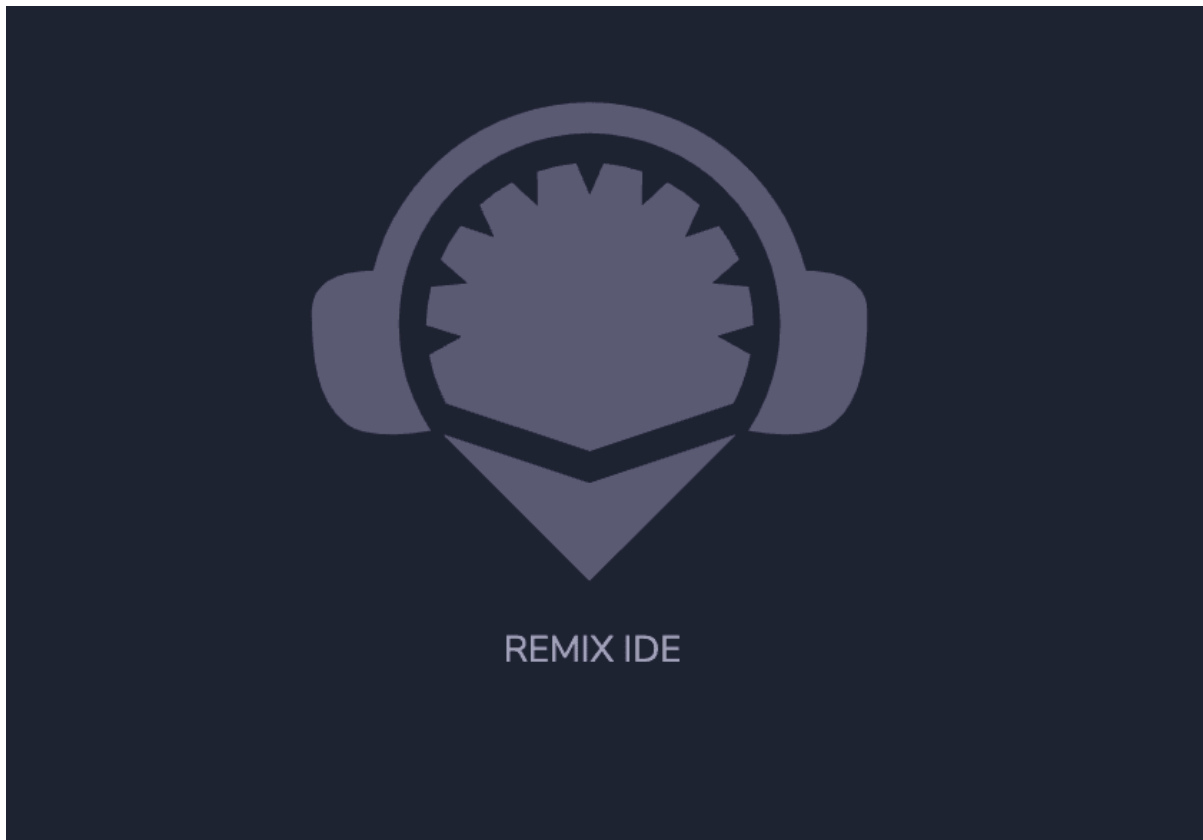


Web 3.0 Unveiled - Day 2

Mentored by FEC

Introduction to Remix IDE



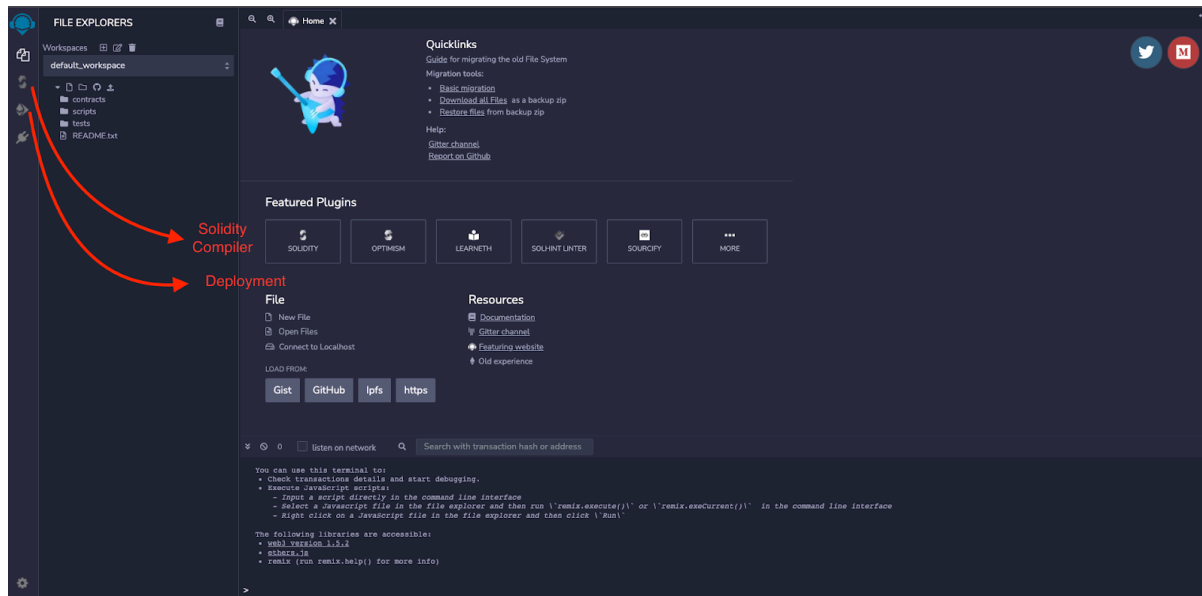
Remix is an open-source, web and desktop [Integrated Development Environment \(IDE\)](https://remix.ethereum.org/) for Ethereum development. It is the easiest development tool to get started with building on Ethereum and has a huge collection of plugins to extend its experience.

Remix helps you write Solidity code directly in the browser and has tools for testing, debugging, and deploying your smart contract to the blockchain.

You can visit Remix at <https://remix.ethereum.org/>

Navigating Remix

When you first open Remix, you will be greeted with a screen like this.



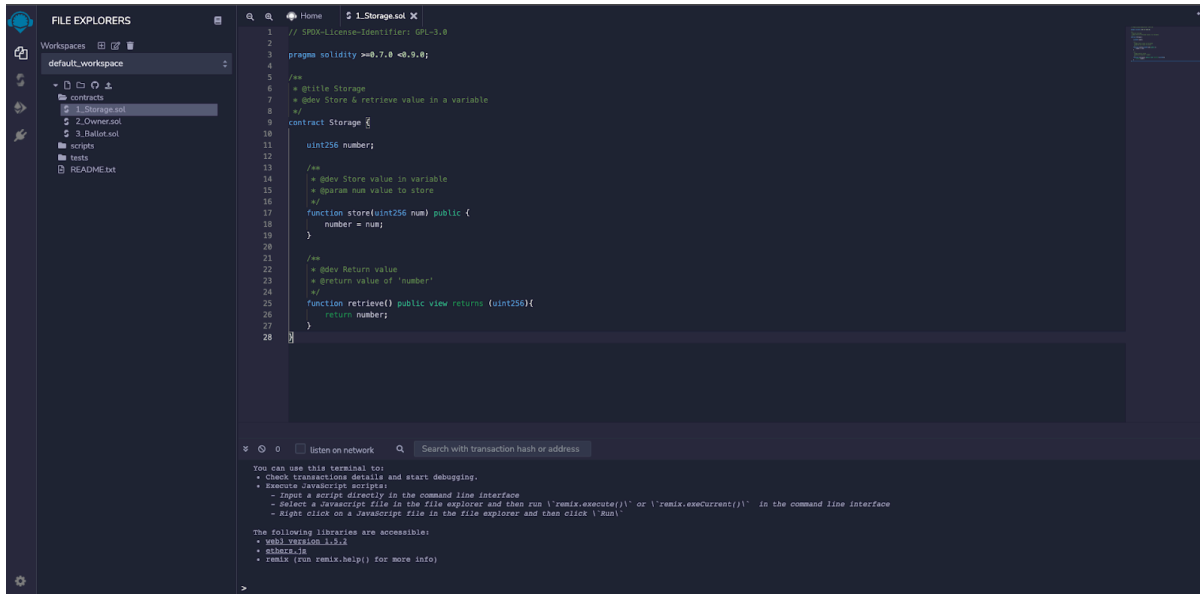
In the left sidebar, you can switch between the **File Explorer**, the **Solidity Compiler**, the **Deployer**, and an **Extensions** panel.

At the bottom, there is an output panel, which displays output from your compilation, your deployments, and your function calls.

In the middle is where you will edit the code. Currently, it displays the home screen of the IDE, but once we open a file, it will become the code editor.

Remix Workflow

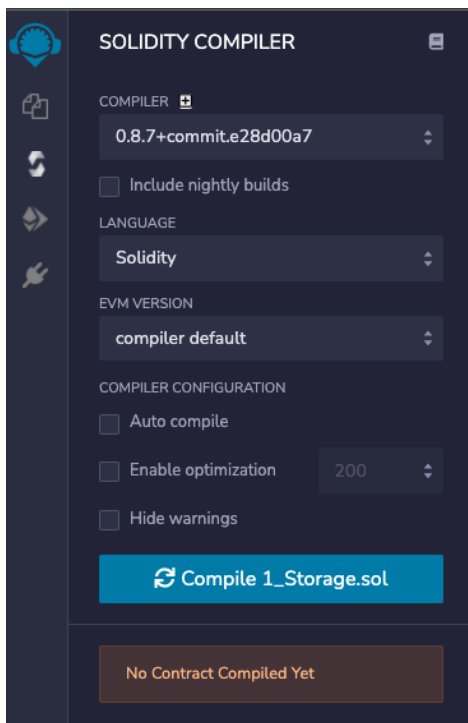
In the sidebar, if you look under the **contracts** folder - Remix ships with three basic smart contracts to help people learn Solidity. Let's take a look at **1_Storage.sol**.



We can see the code editor now.

In the file explorer, we can also see options to create a new file or directory, upload local files, or import files from Github.

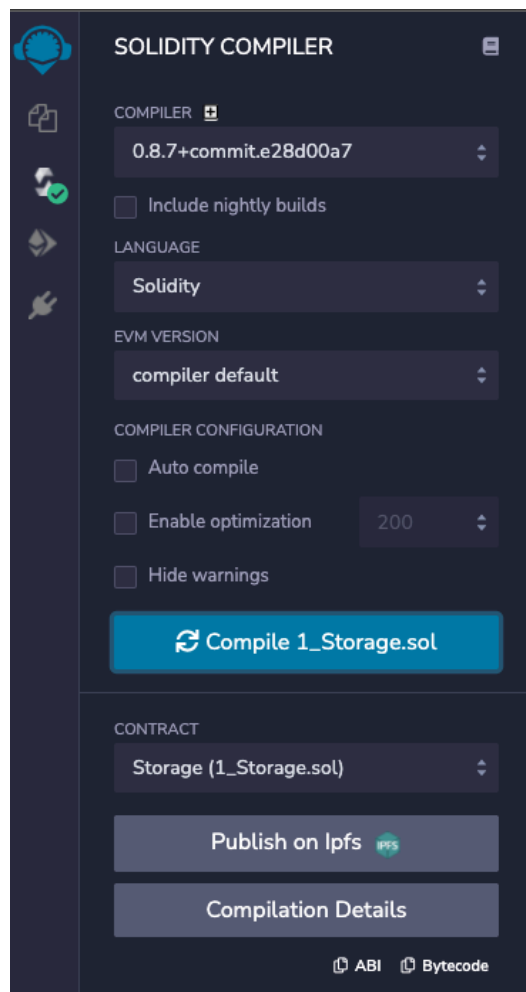
To compile our contracts, we shift over to the `Solidity Compiler` tab, and we will see something like this in the sidebar.



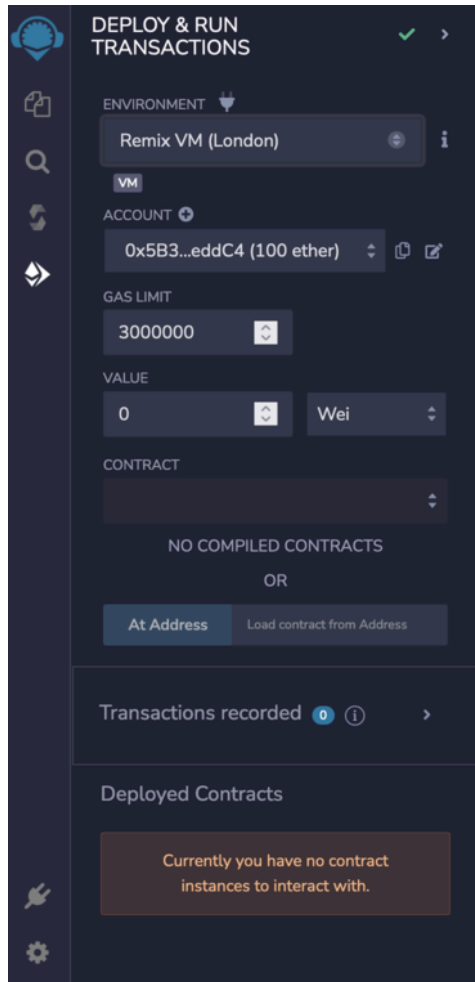
Here, we can choose which **Compiler Version** we want, which smart-contract programming language we are using (mostly, you will just be using Solidity), and some further configuration options.

Note: The other programming language listed in Remix, **Yul**, is a lower-level language. It is meant for intermediate compilation and is closer to the hardware than Solidity. 99% of the time, you will not be coding in Yul. Read more about Yul here - <https://docs.soliditylang.org/en/v0.8.9/yul.html>

Clicking **Compile 1_Storage.sol** will compile the contract and make it ready for deployment.



Moving over to the **Deployment** tab, we will see something like this in the sidebar.



The first thing to note here is the `Environment`. Remix ships with a `Remix VM (London)` - which is a simulator of the Ethereum Virtual Machine (EVM) running the [London Upgrade](#) in the browser. This allows for fast testing and debugging of your smart contract, as long as your contract doesn't depend on another contract deployed to a real Ethereum network. Thankfully, our `Storage` contract does not, so we can test it here in the `Remix VM`.

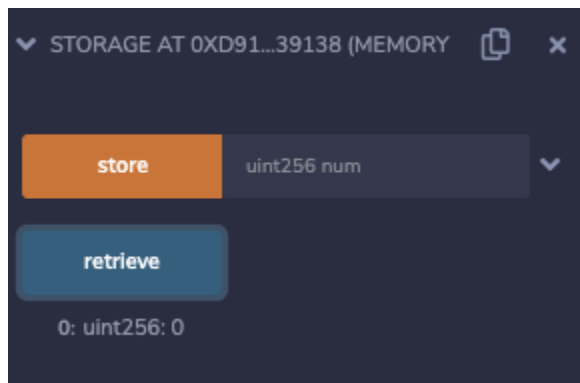
To deploy to actual networks, we will want to change our `Environment` to one of the other options listed there.

Along with the `Remix VM (London)`, Remix creates a set of fake accounts, all loaded up with 100 ETH, to test with.

Select the `1_Storage.sol` contract from the dropdown, and click `Deploy` to deploy the contract.

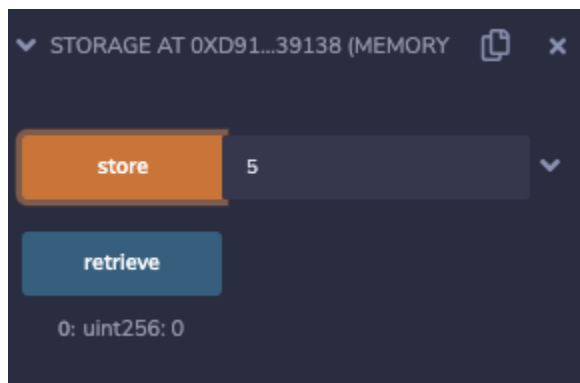
Once the contract is deployed, you will see it under the `Deployed Contracts` section - where you can now call functions on your smart contract.

Calling the `retrieve` function will return a value of `0` right now, which is the default value for integers in Solidity.

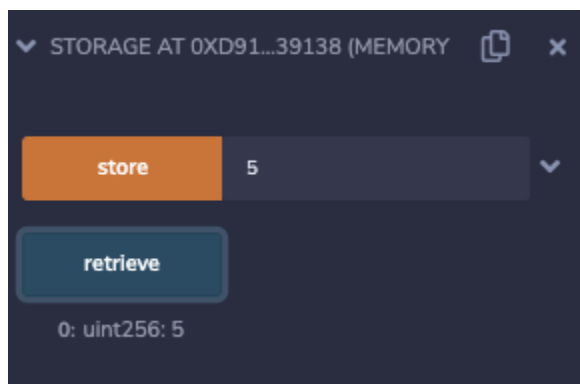


Also, we will see in the Output panel some logs about the call to `Storage.retrieve`, which is our function.

Now, let's try calling the `store` value with the number `5`.



Again, we see some logs in the output panel about the call to `Storage.store`. Now, if we try to `retrieve` again, the output will be `5`.



NOTE - None of these function calls/transactions we made opened up your digital wallet (Metamask). This is because we are testing in the `Remix VM (London)` currently, and that is just a simulator working with fake accounts. When deploying to a real network (Testnet or mainnet), transactions need to be confirmed and signed through your digital wallet.